



Sichuan University -
Pittsburgh Institute

ECE0202 Embedded Processors & Interfacing + Lab

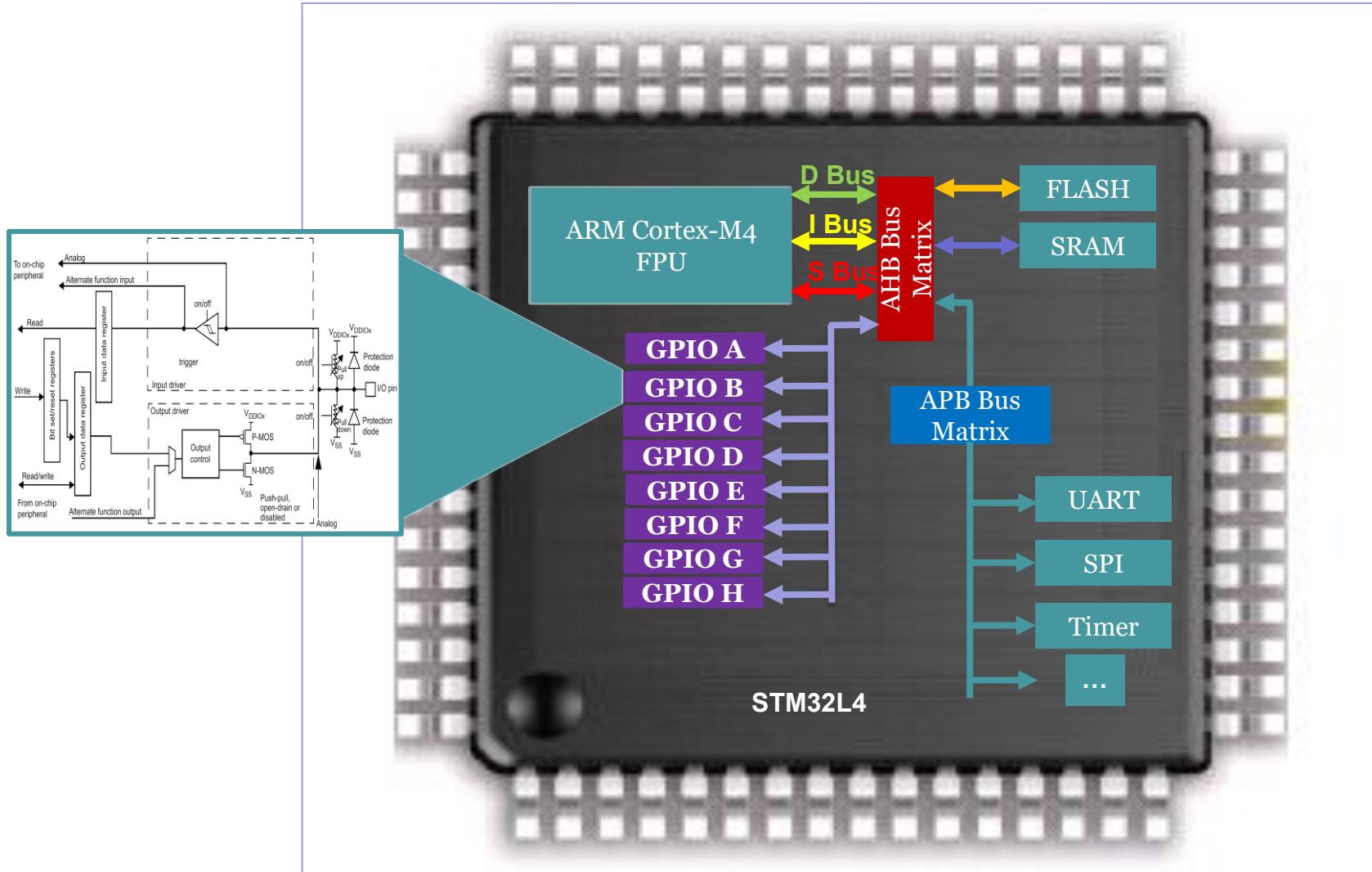
Sichuan University-Pittsburgh Institute



Topics

- Steps to Read Data from GPIO Pin

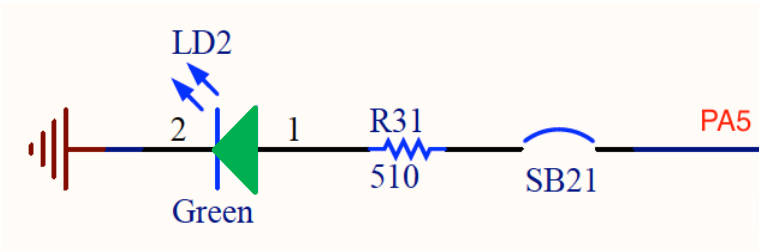
I/O Peripherals



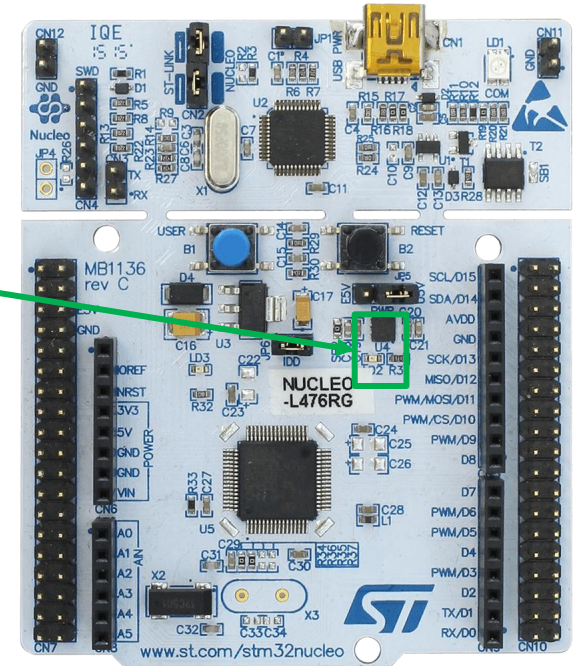
Green LED (PA5)



PA5

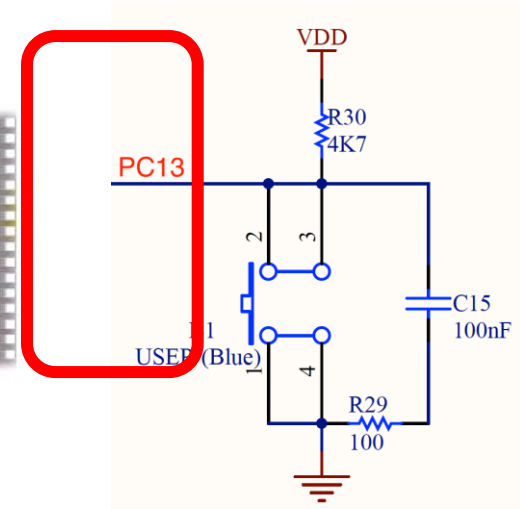
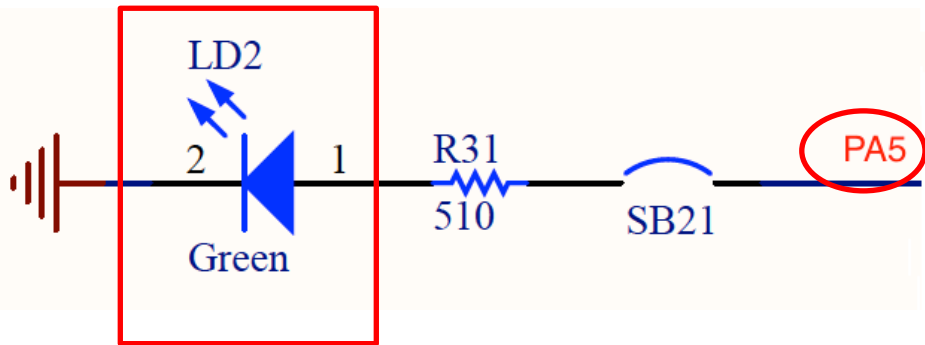


Green
LEDs



PA5	Green LED
High	On
Low	Off

GPIO physical pins

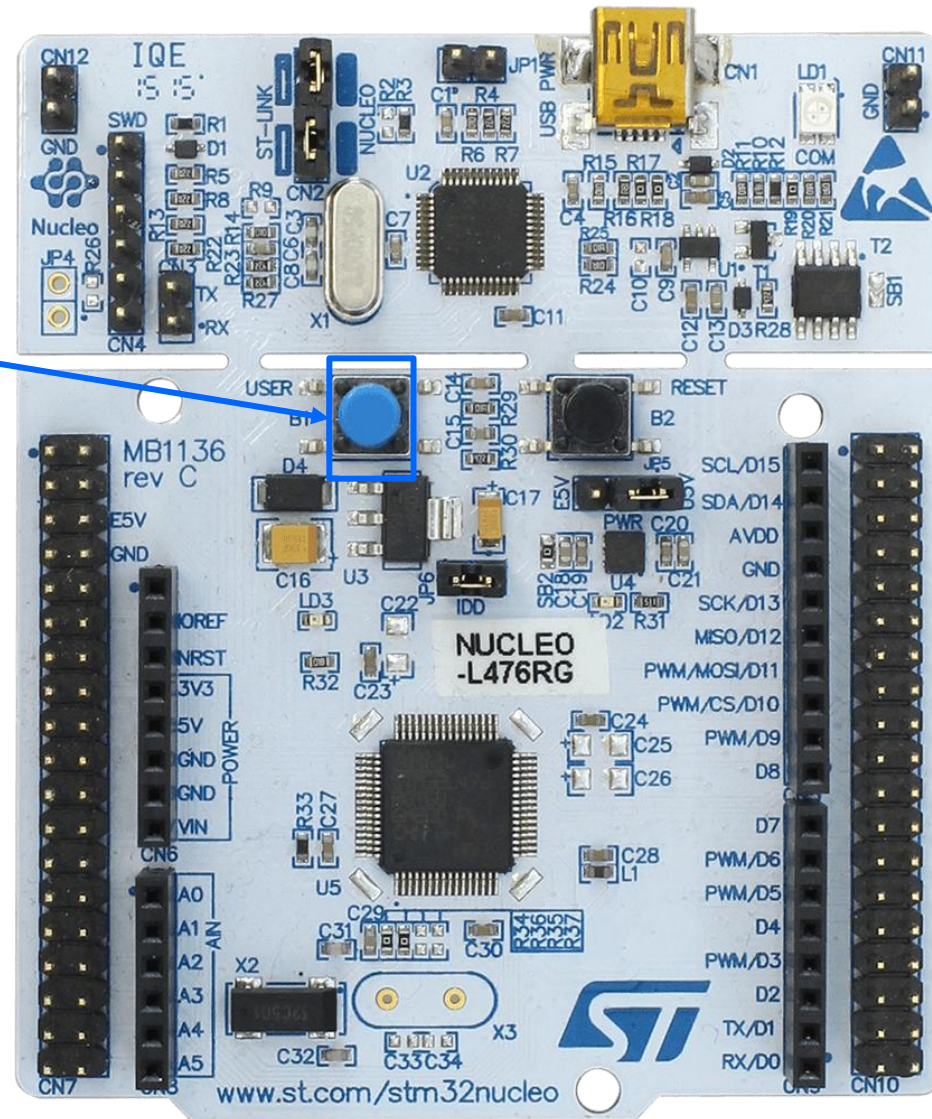
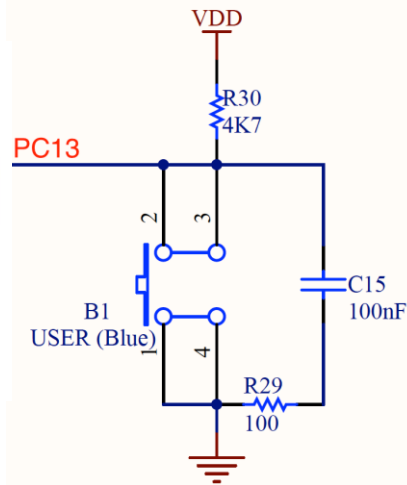




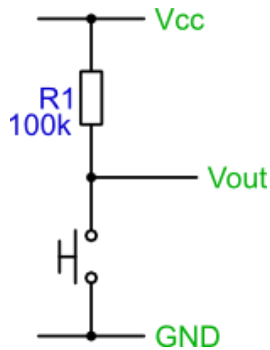
Steps to Read Data from GPIO Pin

Push Button

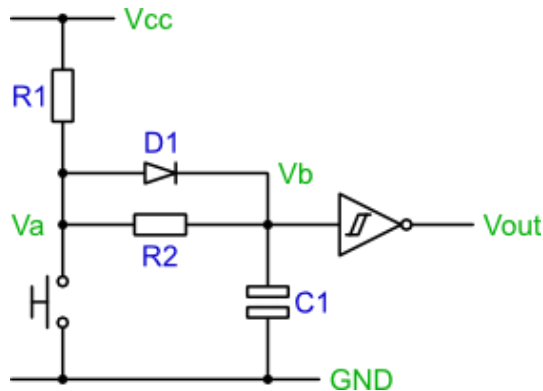
Push Button



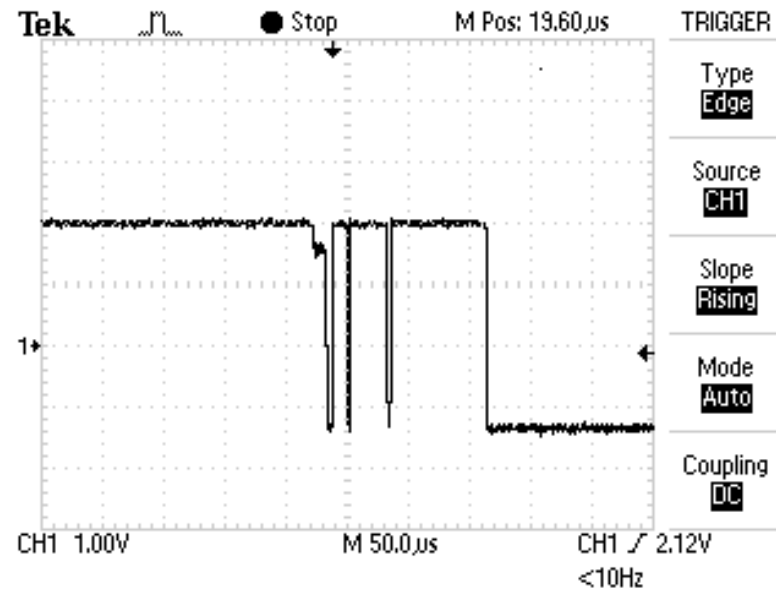
Switch Bounce



Simple switch pull-up circuit



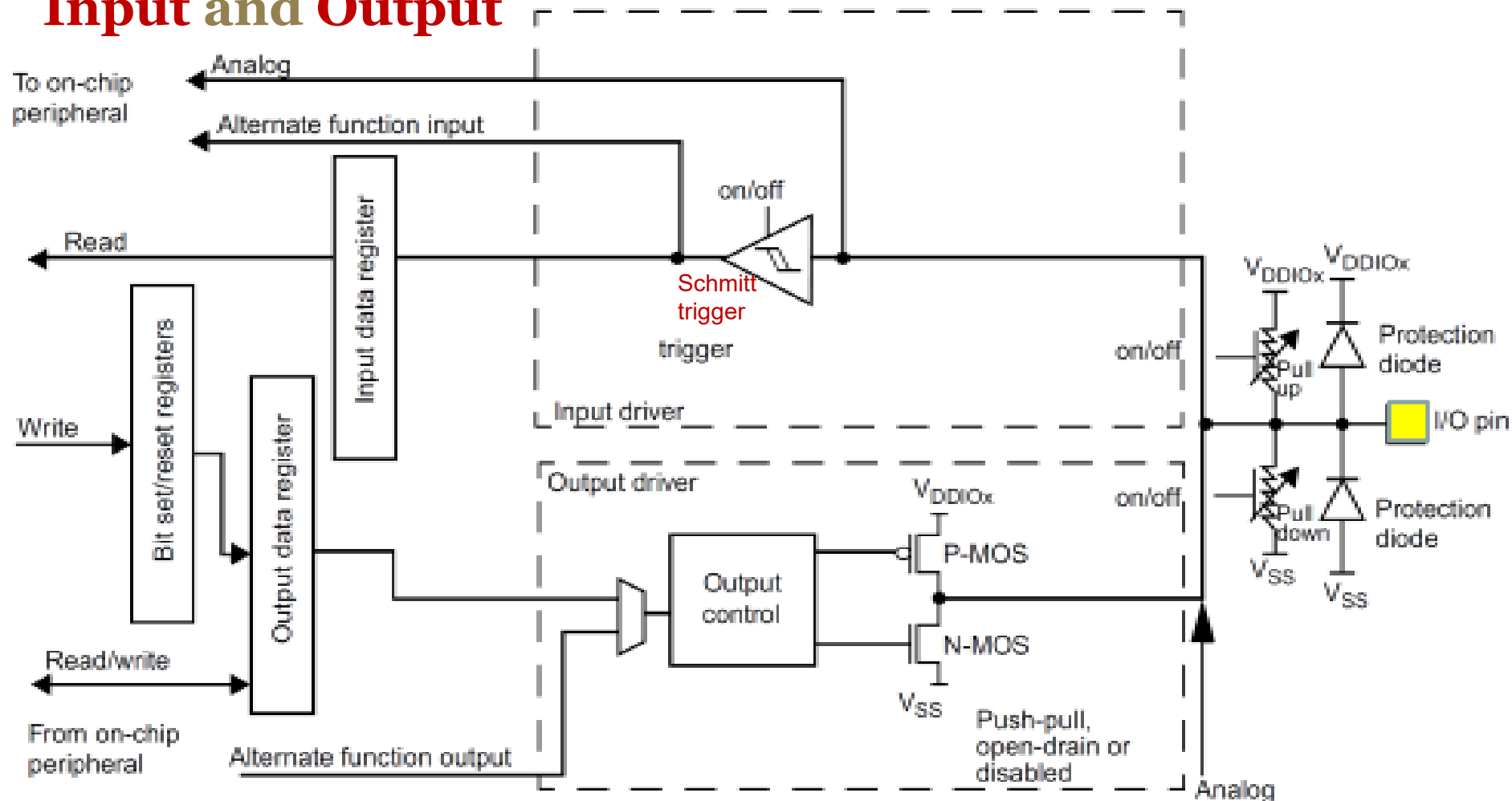
A Switch debouncing circuit



Switch bounce produced on switch press

Basic Structure of an I/O Port Bit

Input and Output

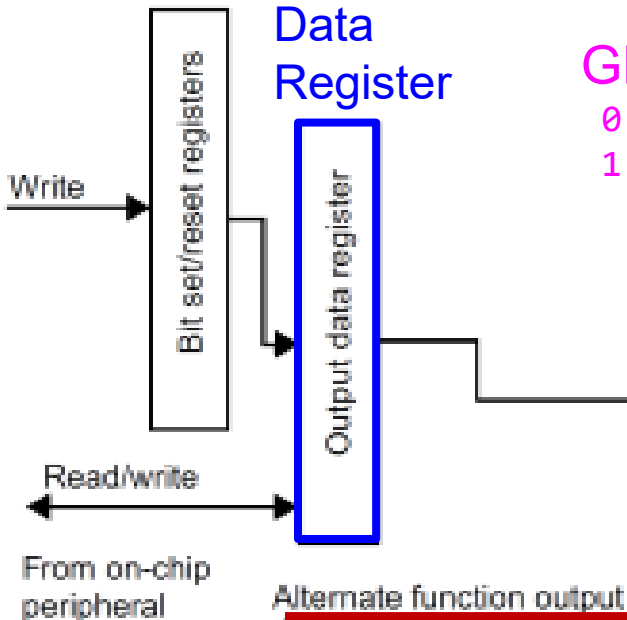


Basic Structure of an I/O Port Bit:

GPIO Pull-up/Pull-down Register (PUPDR)

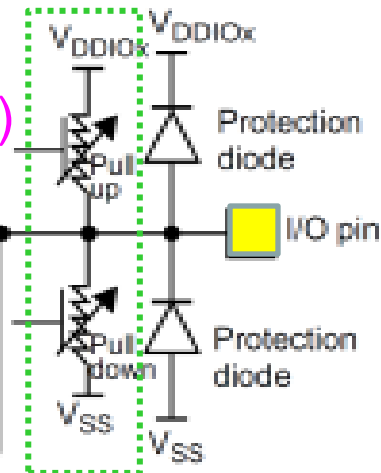
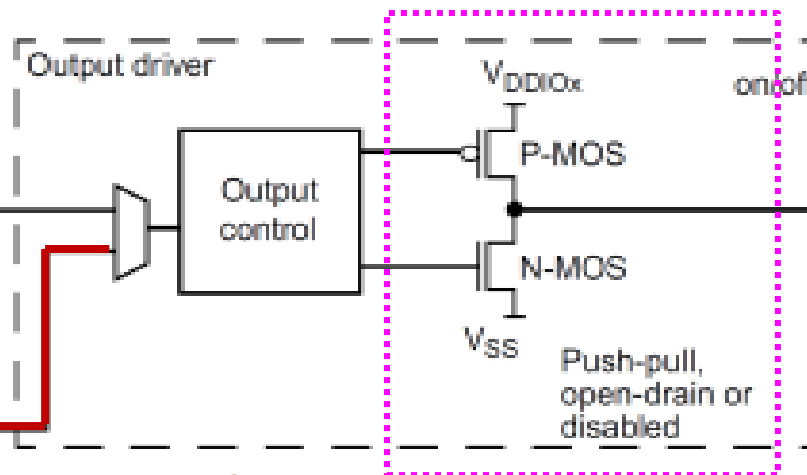
00 = No pull-up, pull-down 01 = Pull-up
10 = Pull-down 11 = Reserved

Output Data Register



GPIO Output Type Register (OTYPER)

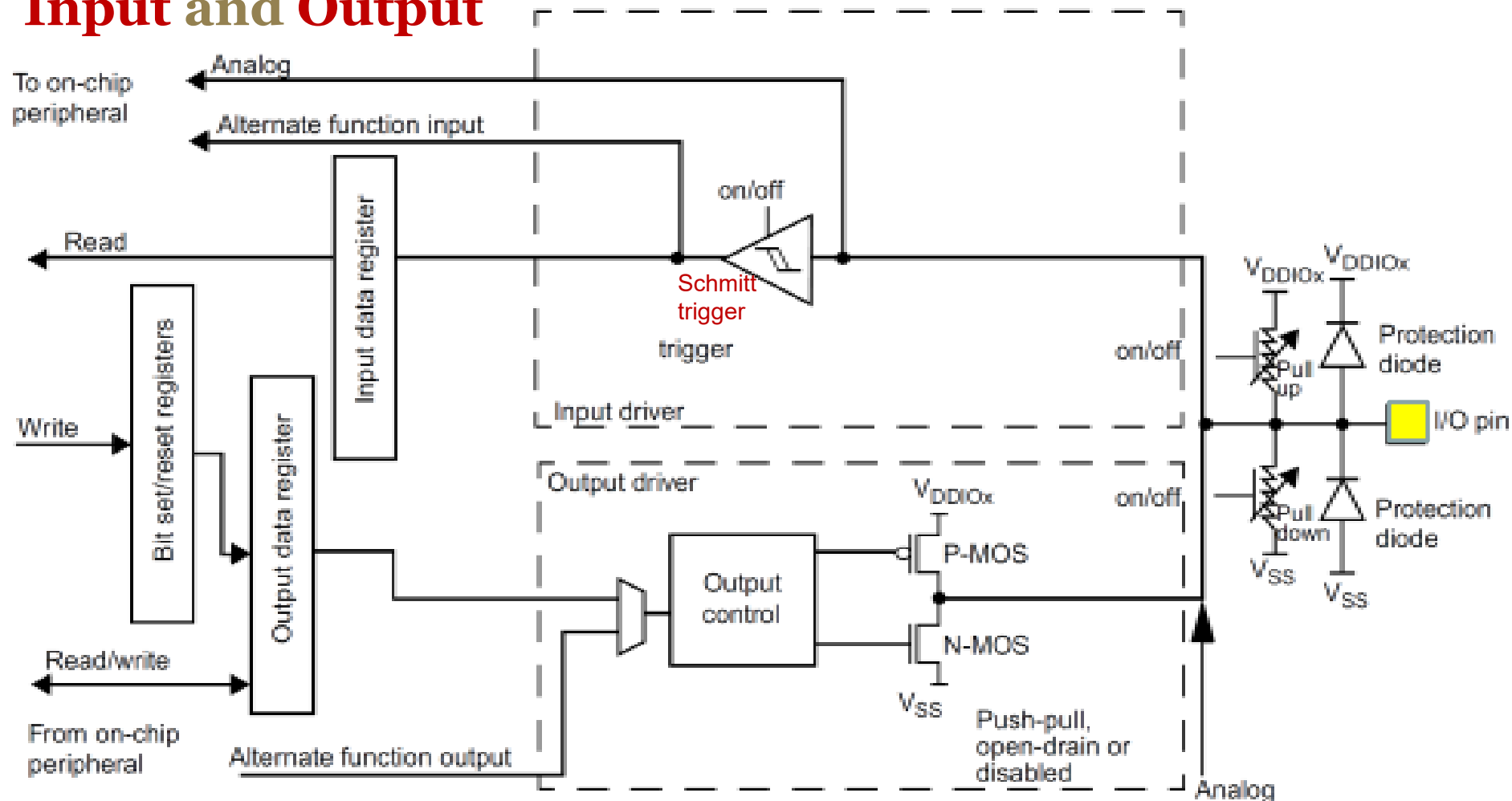
0 = Output push-pull (default)
1 = Output open-drain



GPIO MODE Register 00 = Input, 01 = Output,
 10 = AF, 11 = Analog (default)

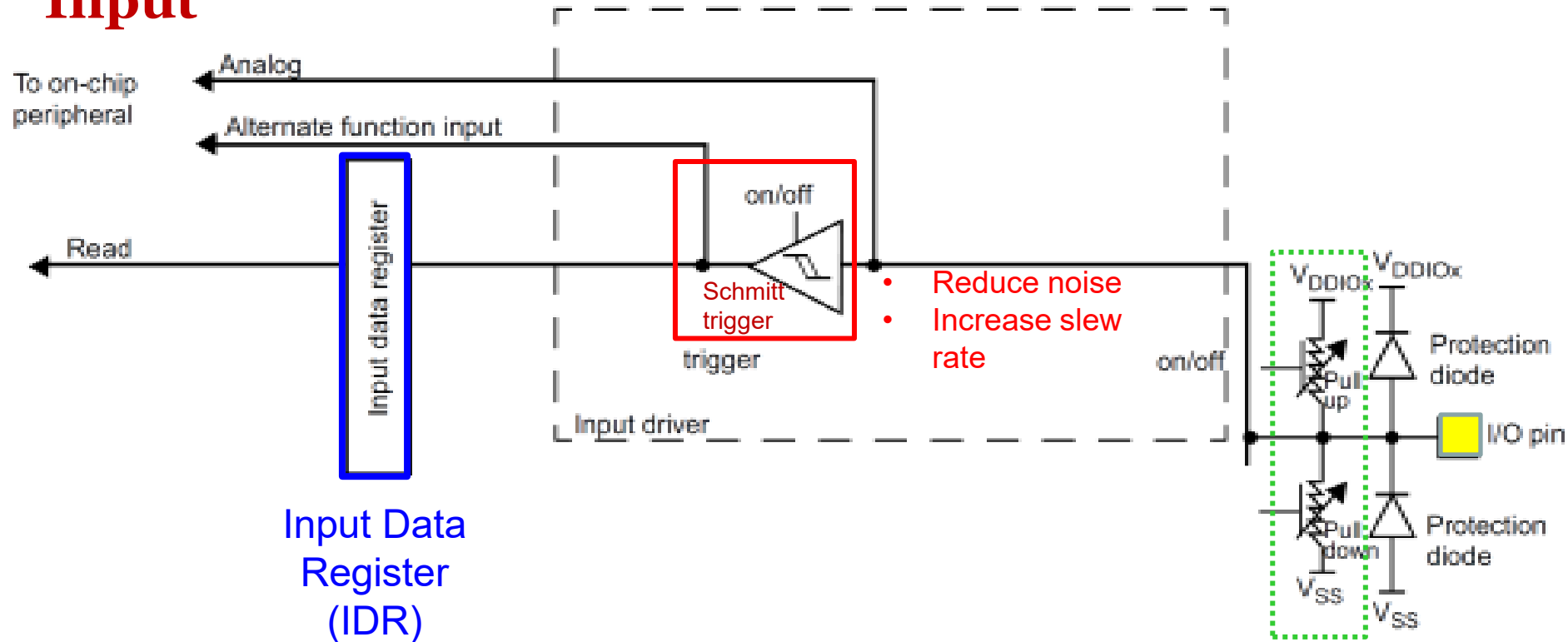
Basic Structure of an I/O Port Bit

Input and Output



Basic Structure of an I/O Port Bit:

Input



GPIO Pull-up/Pull-down Register (PUPDR)

00 = No pull-up, pull-down 01 = Pull-up
10 = Pull-down 11 = Reserved

Enable Clock

- AHB2 peripheral clock enable register (RCC_AHB2ENR)

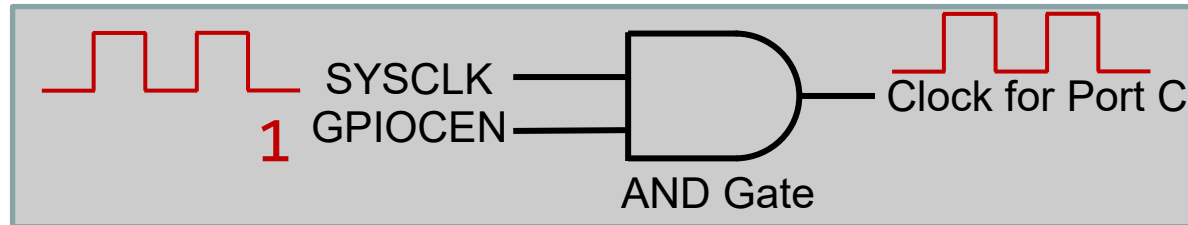
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RNG EN	Res.	AESEN
													rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	ADCEN	OTGFS EN	Res.	Res.	Res.	Res.	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
		rw	rw					rw	rw	rw	rw	rw	rw	rw	rw

Bit 0 **GPIOCEN**: IO port C clock enable

Set and cleared by software.

0: IO port C clock disabled

1: IO port C clock enabled



```
#define RCC_AHB2ENR_GPIOCEN ((uint32_t)0x00000004U) 00000100B
```

```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOCEN; // Enable clock of Port C
```


GPIO Mode Register (**MODER**)

- 32 bits (16 pins, 2 bits per pin)

Pin 13

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y+1:2y **MODEy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O mode.

00: Input mode

01: General purpose output mode

10: Alternate function mode

11: Analog mode (reset state)

```
// Set Pin 13 as input
```

```
GPIOC->MODER &= ~(3UL<< 26); // Clear bits 1 and 2 for Pin 0
```

GPIO Pull-up/Pull-down Register (**PUPDR**)

- 16 pins per port, 2 bits per pin

Pin 13

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPD15[1:0]		PUPD14[1:0]		PUPD13[1:0]		PUPD12[1:0]		PUPD11[1:0]		PUPD10[1:0]		PUPD9[1:0]		PUPD8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPD7[1:0]		PUPD6[1:0]		PUPD5[1:0]		PUPD4[1:0]		PUPD3[1:0]		PUPD2[1:0]		PUPD1[1:0]		PUPD0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits $2y+1:2y$ **PUPDy[1:0]**: Port x configuration bits ($y = 0..15$)

These bits are written by software to configure the I/O pull-up or pull-down

00: No pull-up, pull-down

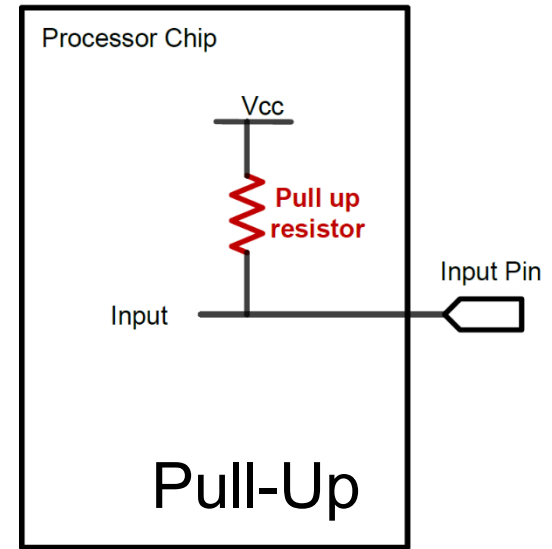
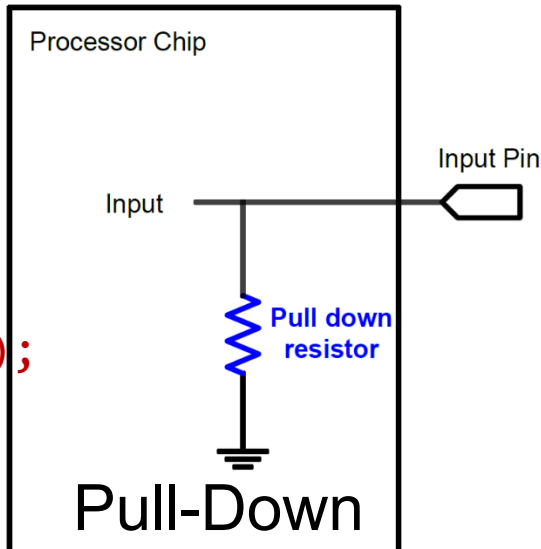
01: Pull-up

10: Pull-down

11: Reserved

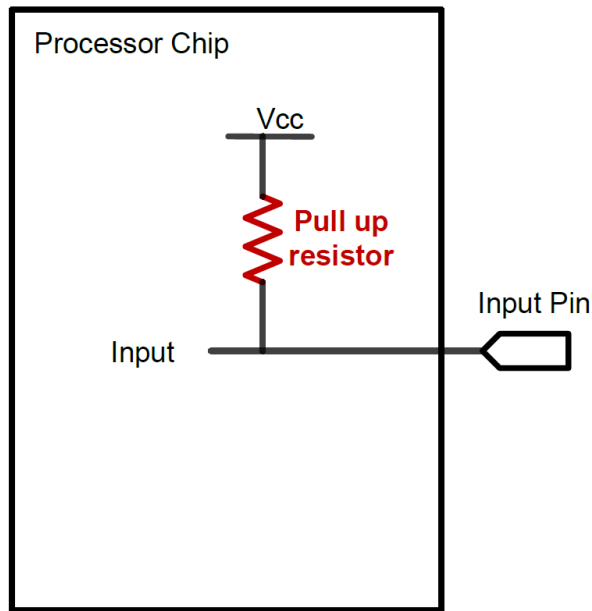
// No pull-up, pull-down

GPIOC->PUPDR &= ~(3UL<< 26);



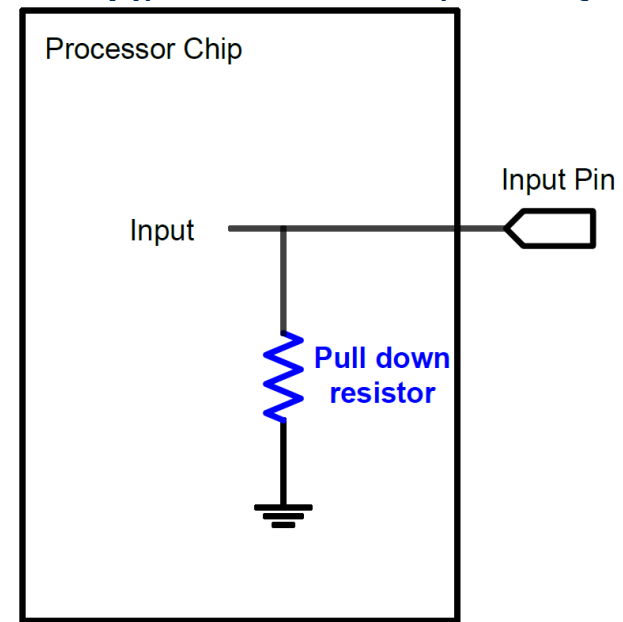
GPIO Input: Pull Up and Pull Down

- A digital input can have three states: High, Low, and High-Impedance (also called floating, tri-stated, HiZ)



Pull-Up

If external input is HiZ, the input is read as a valid HIGH.



Pull-Down

If external input is HiZ, the input is read as a valid LOW.

GPIO Input Data Register (IDR)

- ▶ 16 bits reserved, 16 data bits (1 bit per pin)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

```
// Demo of reading pin 13
uint32_t mask = 1UL<<13;
uint32_t input = (GPIOC->IDR & mask) == mask;
```

or

```
uint32_t input = (GPIOA->IDR & mask) >> 13;
```

Read Input of Pin PC.13

```
uint32_t input;

RCC->AHB2ENR |= RCC_AHB2ENR_GPIOCEN; // Enable clock of Port C

GPIOC->MODER &= ~(3UL << 26); // Set PC.13 as digital input

GPIOC->PUPDR &= ~(3UL << 26); // No pull-up, no pull-down

// Read pin 13
input = (GPIOC->IDR & 1UL<<13);

if (input == 0) { // Push Button is pressed
    ...
}
else { // Push Button is not pressed
    ...
}
```

Read Input of Pin PC.13

```
uint32_t input;

RCC->AHB2ENR |= RCC_AHB2ENR_GPIOCEN; // Enable clock of Port C

GPIOC->MODER &= ~(3UL << 26); // Set PC.13 as digital input

GPIOC->PUPDR &= ~(3UL << 26); // No pull-up, no pull-down

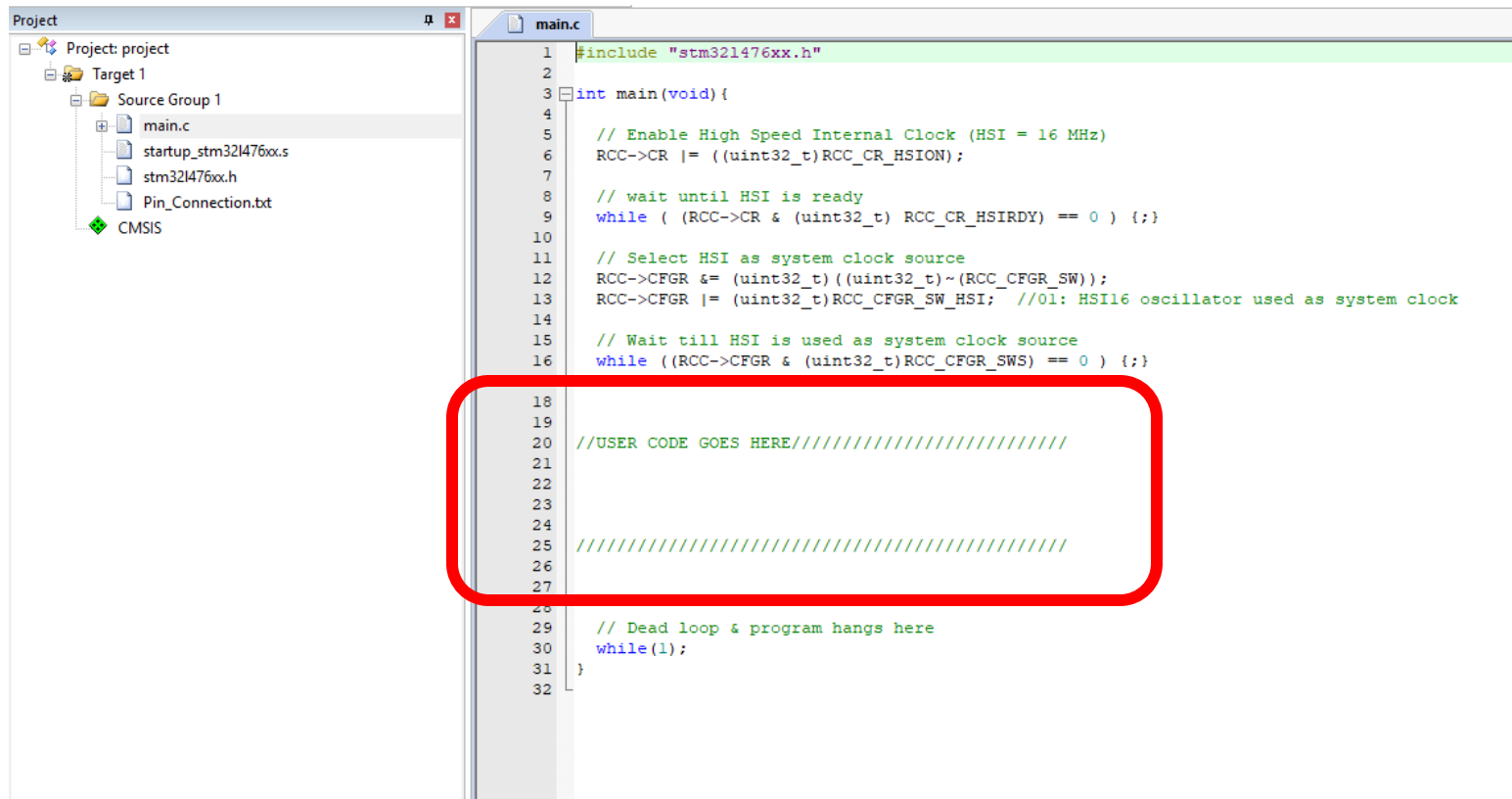
While(1){ // Read pin 13
    input = (GPIOC->IDR & 1UL<<13);

    if (input == 0) { // Push Button is pressed
        ...
    }
    else { // Push Button is not pressed
        ...
    }
}
```


C language project template

- Download C project template
- Includes a generic “main.c” with some setup functions

Main.c



```
1  #include "stm321476xx.h"
2
3  int main(void) {
4
5      // Enable High Speed Internal Clock (HSI = 16 MHz)
6      RCC->CR |= (uint32_t)RCC_CR_HSION;
7
8      // wait until HSI is ready
9      while ( (RCC->CR & (uint32_t) RCC_CR_HSIIRDY) == 0 ) {}
10
11     // Select HSI as system clock source
12     RCC->CFGR &= (uint32_t) ((uint32_t)~(RCC_CFGR_SW));
13     RCC->CFGR |= (uint32_t)RCC_CFGR_SW_HSI; //01: HSI16 oscillator used as system clock
14
15     // Wait till HSI is used as system clock source
16     while ((RCC->CFGR & (uint32_t)RCC_CFGR_SWS) == 0 ) {}
17
18
19
20     //USER CODE GOES HERE////////////////////////////////////
21
22
23
24
25     //////////////////////////////////////
26
27
28
29     // Dead loop & program hangs here
30     while(1);
31 }
32
```